

Progress report 1 of 10

LLM-Guided Neural Network Compilation for Real-time UAV Edge Inference

Atul Kumar Gupta · 25MCSS06

Why the companion computer is a compile problem

This is the first progress note.

A small drone splits the work. PX4 on the flight controller keeps the aircraft in the air. A Linux computer beside it has to run the camera network, and it has to keep up with the frames.

A compile recipe is the set of choices that turn a trained network into a file that can run: which joins of operations to allow, whether weights are 8-bit, and how many CPU threads to use. People still pick that recipe by habit. A recipe that is fast on a desktop GPU is often slow, or numerically wrong, on a small ARM CPU.

The lab stack had to be named before any compiler was written.

- The companion computer runs the camera network. The flight controller does not.
- PX4 is the flight stack, including in simulation.
- Gazebo is the world the aircraft flies in.
- ROS 2 carries messages between programs.
- ONNX stores the network as a graph of operations.

A camera that sends a frame every 33 ms does not leave much time. If the network, and the compile choices around it, use more than that, frames are dropped. That is why the recipe is part of the thesis, not a setup detail.

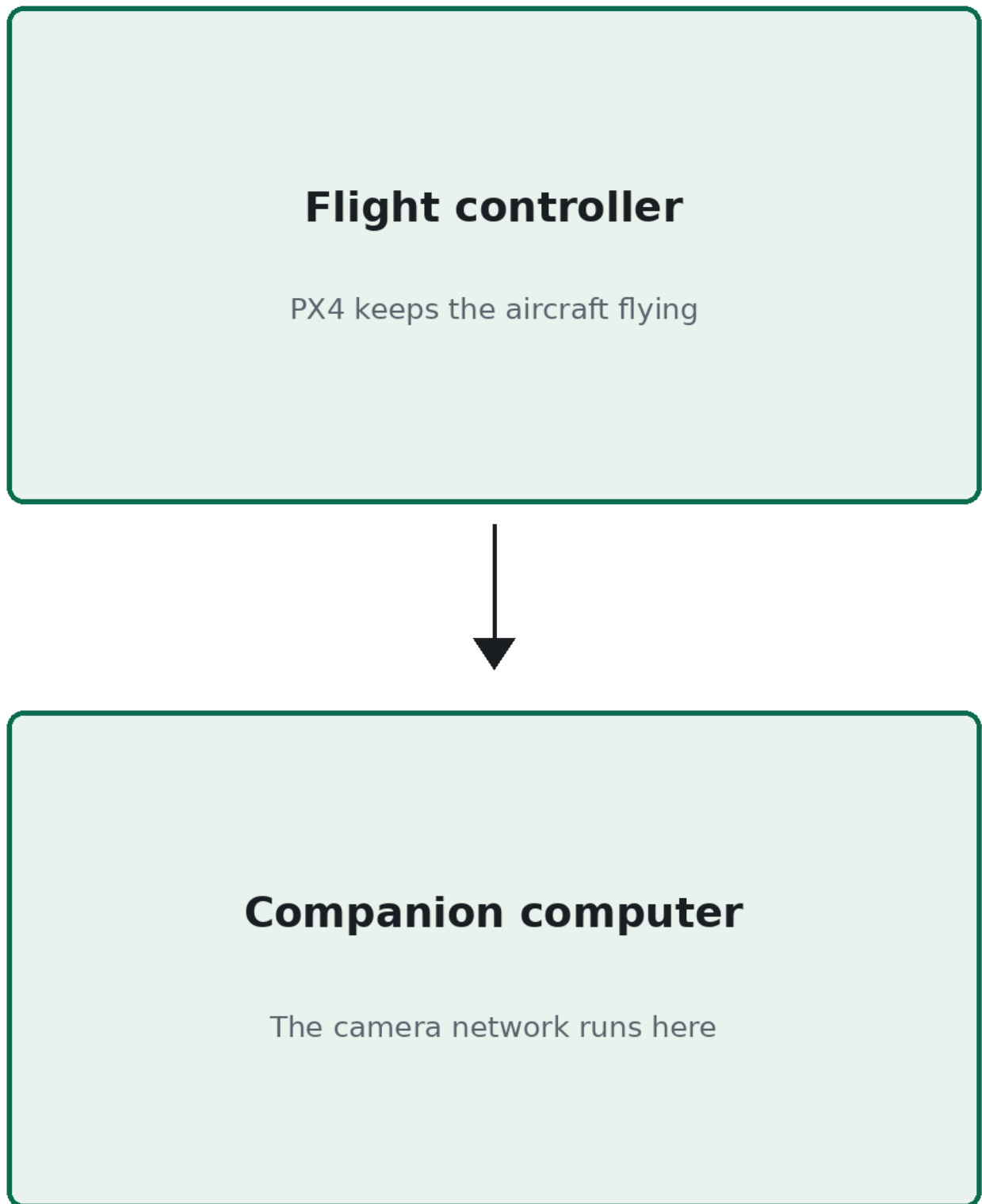


Figure 1. Flight stays on PX4. The camera network stays on the companion computer.

Flight and vision are separate jobs. The next note asks who is allowed to choose the compile recipe.